

Assignment 1 DISTRIBUTED COMPUTING

Henri Junior Houphouet

1 Introduction

In this assignment, we study how jobs are processed in multiple queues and compare the performance of different load balancing strategies, especially the supermarket (power-of-d choices) method.

First, we analyze the system under the memoryless assumption, where both job arrival times and service times follow exponential distributions. We compare the simulation results with the theoretical results studied in class.

Then, we extend the analysis by using Weibull distributions for arrivals and service times. This allows us to study non-memoryless workloads and understand how different workload characteristics affect the system performance.

Finally, we analyze an extra extension added to the simulator and discuss its impact on performance and the trade-offs it creates.

2 Experiments under memoryless workloads

We start by testing the simulator with $d = 1$, which represents the random assignment strategy. The simulation results for the queue length distribution are very close to the theoretical predictions, confirming that the simulator is correct.

After that, we perform the same experiments with higher values of d ($d = 2, 5, 10$). We observe that larger values of d greatly reduce the chance of long queues appearing, showing the advantage of the supermarket approach (power-of-d choices). For instance, at $\lambda = 0.95$, the fraction of queues with at least 4 jobs drops from 0.81 with $d = 1$ to 0.48 with $d = 2$, 0.04 with $d = 5$, and 0.016 with $d = 10$. Most of the improvement is obtained when going from $d = 1$ to $d = 2$, while further increases bring only marginal gains.

The agreement between the experimental and theoretical results is excellent for moderate loads ($\lambda \leq 0.95$). At $\lambda = 0.99$, the experimental curve lies slightly above the theoretical one. This is expected: the theoretical formula is an asymptotic result valid for large n and long simulation time, so finite simulations can only approximate it.

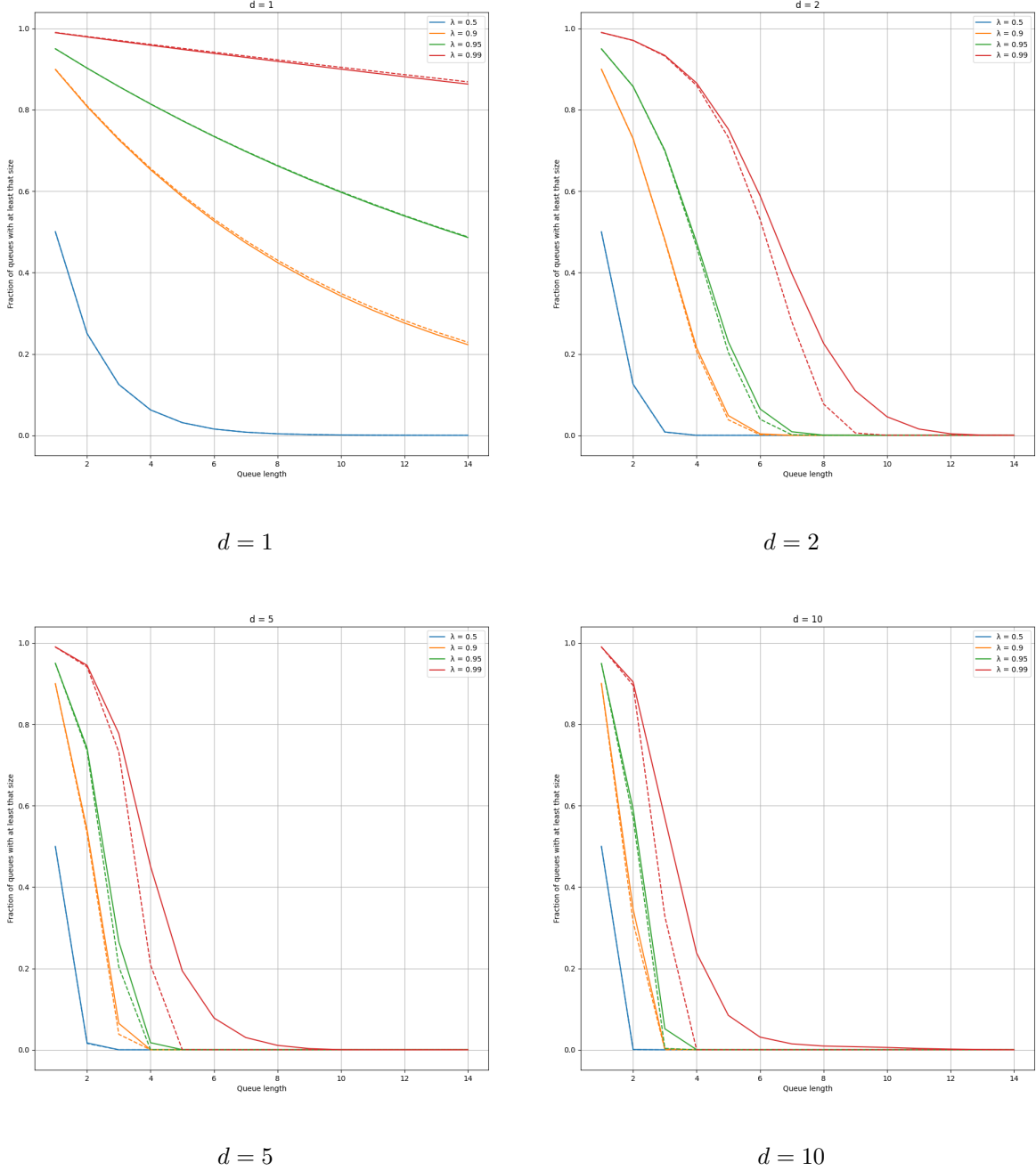


Figure 1: Simulation vs theory for different values of d

3 Experiments under Weibull

We tested three values of the Weibull shape parameter k applied to service times ($k \in \{0.5, 1.0, 2.0\}$), keeping inter-arrival times exponential ($k_{\text{arrival}} = 1$) and $d = 2$. This isolates the effect of service time variability on queue lengths. With $k = 1$, we recover the exponential distribution: results closely match the theoretical prediction, confirming our implementation is correct. With $k = 2$, inter-arrival times and service durations are more concentrated around their mean, making the workload more predictable. In this case, queues are actually shorter than the exponential theory predicts — the solid curves drop faster than the dashed ones because the absence of very long jobs prevents servers from getting stuck. With $k = 0.5$ (heavy-tailed), the behavior degrades dramatically: a small number of very large jobs monopolize servers, and bursts of arrivals pile up faster than servers can respond. With $\lambda = 0.95$, over

6% of queues still have at least 14 jobs, compared to essentially 0% in theory; at $\lambda = 0.99$, that fraction is still 58% at length 14. This shows that the memoryless assumption is optimistic: real workloads, which tend to be heavy-tailed, perform significantly worse than the theoretical model predicts, and the gap between simulation and theory grows with load.

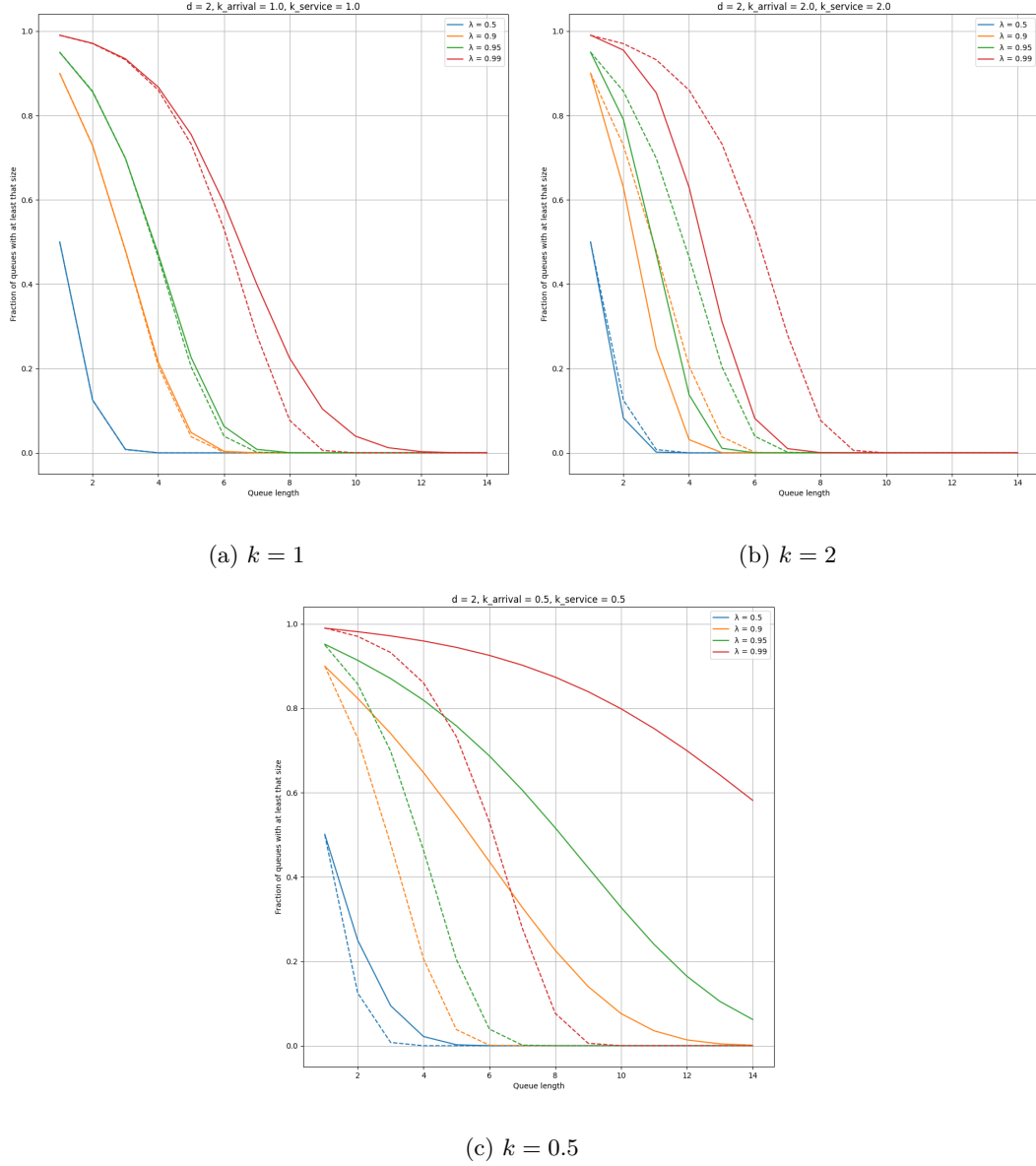


Figure 2: Queue length distribution for the supermarket model with $n = 100$.

4 Least Work Left vs Shortest Queue Under Weibull Workloads

In this assignment, the base dispatcher uses the Shortest Queue (SQ) policy: when a job arrives, it samples d queues at random and joins the one with the fewest jobs. This works well when all jobs take roughly the same time, but in practice job sizes can vary significantly — some jobs are very short while others are extremely long. In such cases, SQ can be misleading: a queue with few jobs might still have a long waiting time if one of those jobs is very large. To investigate this, we implemented an extension called Least Work Left (LWL), where instead of counting jobs, the dispatcher estimates the

total remaining work in each sampled queue and joins the lightest one.

We tested both policies under different load levels ($\lambda \in \{0.5, 0.7, 0.9, 0.95, 0.99\}$) and different Weibull service-time distributions ($k = 0.5$ heavy-tailed, $k = 1.0$ exponential, $k = 2.0$ more uniform). The goal was to see if a better way of assigning jobs improves the system.

The results show that LWL consistently gives lower average waiting time than SQ across all tested configurations. The improvement is most significant under heavy-tailed service times ($k = 0.5$), where job sizes are highly variable. For example, at $\lambda = 0.9$ with $k = 0.5$, LWL reduces the average waiting time significantly compared to SQ. This is because SQ only counts the number of jobs in a queue, and cannot detect queues blocked by a single very large job. LWL avoids this by estimating the total remaining work, and therefore makes better routing decisions when job sizes are uneven. Under $k = 1.0$ and $k = 2.0$, where job sizes are more uniform, the gap between SQ and LWL narrows, confirming that LWL's advantage is directly tied to workload variability.

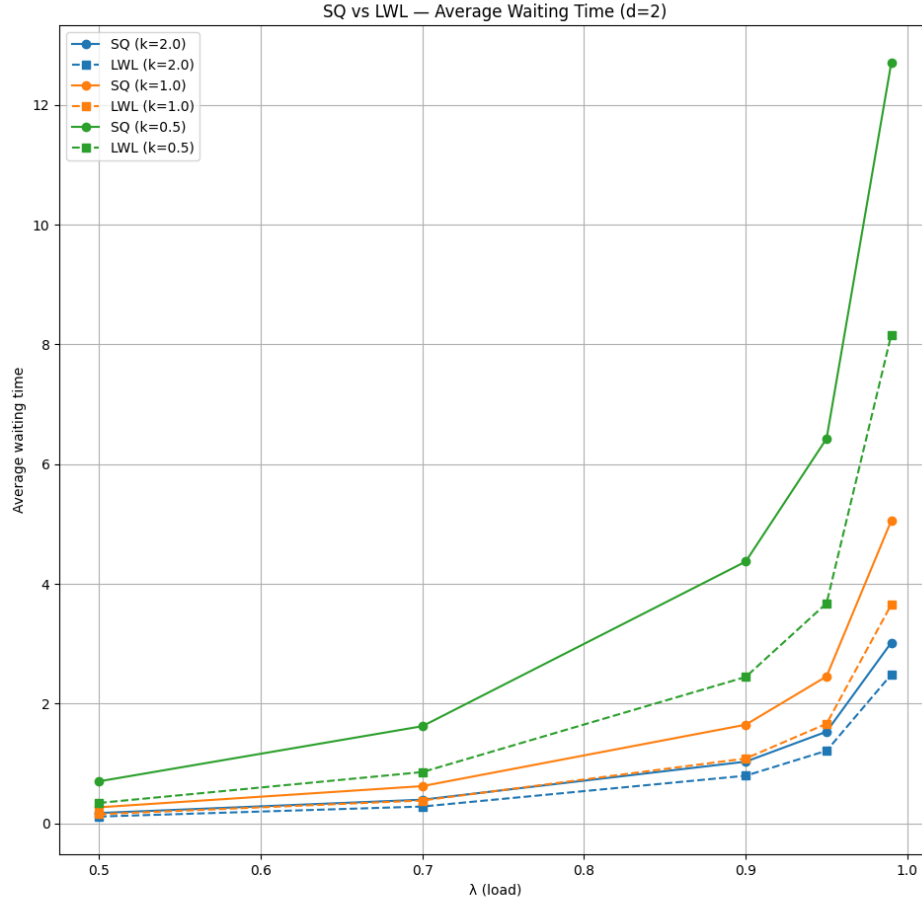


Figure 3: Average waiting time of SQ (solid lines) and LWL (dashed lines) for $d = 2$, $n = 100$, across load levels $\lambda \in \{0.5, 0.7, 0.9, 0.95, 0.99\}$ and service time shapes $k \in \{0.5, 1.0, 2.0\}$. LWL consistently outperforms SQ, with the largest gap at $k = 0.5$.

5 Use of External Resources

This assignment was completed individually.

I used ChatGPT as a support tool to help me understand the simulator code, and as a guide during the implementation of the code and its extension. It also helped me structure and write parts of the report. Only the plotting part was fully generated by ChatGPT.

6 Appendix

This section provides a brief overview of the functionality of each file:

- `discrete_event_sim.py` — The base simulation engine provided for the assignment. Defines the `Simulation` and `Event` classes. No parameters.
- `workloads.py` — Provided utility file. Contains `weibull_generator` and `parse_mustang`. No parameters.
- `m_queue_sim.py` — Supermarket queue model with exponential (memoryless) distributions. Tasks 1 and 2.
 - `--d` : number of queues sampled per arrival (default: 1)
 - `--n` : number of servers (default: 1)
 - `--max-t` : simulation time (default: 1,000,000)
 - `--lamdb` : arrival rate λ (default: 0.7)
 - `--mu` : service rate μ (default: 1)
 - `--seed` : random seed for reproducibility (optional)
 - `--verbose` : enable logging output (optional)
- `run_memoryless.sh` — Runs `m_queue_sim.py` with $d \in \{1, 2, 5, 10\}$, $n = 100$, `max-t` = 100000.
- `w_queue_sim.py` — Extends `m_queue_sim.py` with Weibull distributions. Task 3. Accepts all parameters of `m_queue_sim.py`, plus:
 - `--weibull-shape-arrival` : Weibull shape k for inter-arrival times (default: 1.0, i.e. exponential)
 - `--weibull-shape-service` : Weibull shape k for service times (default: 1.0, i.e. exponential)
- `run_weibull.sh` — Runs `w_queue_sim.py` with $d = 2$, $n = 100$, `max-t` = 100000, across service shapes $k \in \{0.5, 1.0, 2.0\}$.
- `ext_queue_sim.py` — Implements the LWL vs. SQ extension. Accepts all parameters of `w_queue_sim.py`, plus:
 - `--d` : number of queues sampled (default: 1)
 - Routing policy (SQ vs. LWL) is selected internally — both are run and compared automatically.
 - Service shapes $k \in \{0.5, 1.0, 2.0\}$ and loads $\lambda \in \{0.5, 0.7, 0.9, 0.95, 0.99\}$ are fixed internally for the comparison.
- `run_extension.sh` — Runs `ext_queue_sim.py` with $d \in \{1, 2, 5, 10\}$, $n = 100$, `max-t` = 100000.